Technische Grundlagen: Redundanz · Diversität · Wiederherstellbarkeit · Adaptivität

Vorlesungsskript · Sitzung 3 · Grundlagen der Resilienz

Prof. Dr. Dirk Loomans

29. September 2026

Vorbemerkung: Dieses Skript dient als Vorlesungsbegleitung und erhebt keinen akademischen Anspruch. Dennoch kann es bei der Klausurvorbereitung hilfreich sein. Kommentare und Verbesserungsvorschläge gerne an den Autor über dirk.loomans@hs-mainz.de.

Die mit „Hinweis zur Aktualität“ und „Ergänzung zur Vorlesung“ markierten Kästen sowie die Abbildungen wurden bei der Überführung ins Quarto-Projekt ergänzt. Der übrige Text entspricht dem Skript aus dem Wintersemester 2025/26; korrigiert wurden nur Tipp- und Umbruchfehler.

1. Einleitung und Zielsetzung der Sitzung

Die vorangegangene Sitzung behandelte Modelle und Messgrößen zur Bewertung der Resilienz von IT-Systemen (u. a. RTO, RPO, MTTR, MTBF) sowie die entsprechenden normativen Rahmen in NIST SP 800-160 Vol. 2, ISO/IEC 27031 und BSI-Standard 2004. In dieser Sitzung steht nicht mehr die Bewertung, sondern die **Gestaltung** von Resilienz im Mittelpunkt: Wie werden Systeme technisch so gebaut, dass sie Störungen nicht nur überstehen, sondern kontrolliert wiederhergestellt werden können — unabhängig davon, ob sie in klassischen Rechenzentren, in der Cloud oder in hybriden Architekturen betrieben werden.

Technische Resilienz folgt vier Prinzipien, die unabhängig von Technologie-Stacks, Betriebsformen und Anbietern gelten: **Redundanz, Diversität, Wiederherstellbarkeit und Adaptivität**. Diese Prinzipien bilden den konstruktiven Gegenpol zu den in Sitzung 2 behandelten Messgrößen: Während Messgrößen Resilienz *sichtbar machen*, erzeugen diese Prinzipien Resilienz *im Design*.

Hybridlandschaften aus On-Premises-Systemen und Cloud-Diensten verschärfen die Anforderungen: Einerseits steigt die technische Resilienzfähigkeit durch elastische Cloud-Mechanismen (Multi-AZ-Deployments, Auto-Scaling, Cross-Region-Failover). Andererseits entstehen neue Abhängigkeiten (z. B. zentrale Identity-Systeme oder Cloud-Provider-Lock-In), die als Single Points of Failure auftreten können. Resilienz muss daher sowohl **in die Architektur eingebettet** als auch **im Betrieb eingeübt** werden.

Ziel dieser Sitzung ist es, technische Resilienz **systematisch aus den vier Architekturprinzipien heraus zu analysieren und auf hybride Szenarien anzuwenden**. Im Anschluss erfolgt eine Übung zur Identifikation und Einordnung von Single Points of Failure (SPOFs) in realen Systembeispielen.

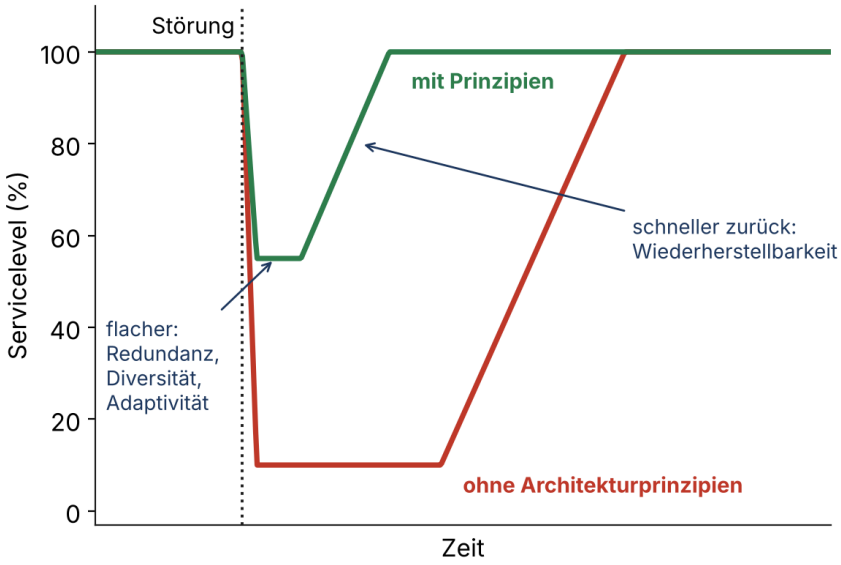


Abbildung 1: Wirkung der Architekturprinzipien auf die Resilienzkurve aus Sitzung 2

2. Prinzip 1: Redundanz — Eliminierung von Einzelbruchpunkten

Redundanz bezeichnet die vorsätzliche Mehrfachvorhaltung kritischer Ressourcen mit dem Ziel, einzelne Komponenten ausfallen zu lassen, ohne dass das Gesamtsystem seine Funktion verliert. Sie ist damit das älteste und zugleich fundamentalste technische Resilienzprinzip.

2.1 Formen von Redundanz in On-Prem-Umgebungen

In klassischen Rechenzentrumsarchitekturen erfolgt Redundanz in der Regel durch Hardware- und Infrastrukturduplikation, u. a.:

- **Stromversorgung** (Dual Power Feed, USV, Diesel-Backup)
- **Netzwerkpfade** (redundante Core-Switches, MPLS-/ISP-Doppelwege)
- **Clusterbildung** (z. B. SQL AlwaysOn, VMware HA)
- **Standortduplikation** — zweites RZ als Hot/Warm/Cold-Standby

Die Ausfallsicherheit steigt proportional zum Grad der Entkopplung. Zwei USVs im gleichen Brandabschnitt sind keine Resilienz, sondern Redundanz ohne Isolation.

2.2 Formen von Redundanz in Cloud-Architekturen

Cloud-Anbieter abstrahieren Redundanz von der physischen Ebene und modellieren sie als logische Deployment-Option:

- **Multi-AZ-Deployment** innerhalb einer Region
- **Load Balancing + Auto-Scaling** zur Kompensation von Knotenverlusten
- **Cross-Region-Failover** zur Absicherung gegen Standortausfälle

Während On-Prem Redundanz kapazitiv vorgehalten werden muss (CapEx), ist sie in der Cloud elastisch und betriebsabhängig konsumierbar (OpEx). Resilienz wird dadurch **kaufbar**, aber nicht automatisch **vorhanden** — sie muss im Architekturentwurf aktiviert werden.

2.3 Grenzen der Redundanz

Redundanz schützt ausschließlich gegen **komponentielle Ausfälle**, nicht gegen:

- logische oder Konfigurationsfehler,
- konzertierte Angriffe (Ransomware, Wiper),
- gemeinsame Abhängigkeiten (z. B. zentrales AD),
- nicht getestete Umschaltprozesse.

NIST SP 800-160 Vol. 2 weist explizit darauf hin, dass Redundanz ohne getestete Transition **keine Resilience Capability**, sondern nur eine **strukturelle Reserve** darstellt.

> Ergänzung zur Vorlesung: British Airways 2017
>
> Am 27. Mai 2017 trennte ein Techniker in einem Rechenzentrum bei Heathrow die Stromversorgung. Beim Wiederzuschalten beschädigte eine Überspannung die IT-Systeme, und die Ausweichsysteme übernahmen nicht. Check-in, Gepäckabfertigung und Flugplanung fielen aus; allein am ersten Tag strich British Airways 479 Flüge, mindestens 75.000 Passagiere waren betroffen, die Kosten wurden auf bis zu 100 Mio. € geschätzt. Warum die Ausweichsysteme nicht griffen, wurde öffentlich nie vollständig geklärt.
>
> Der Fall zeigt zwei Grenzen der Redundanz zugleich: eine gemeinsame Abhängigkeit (die Stromversorgung) und eine Umschaltung, die unter realen Bedingungen nicht funktionierte.
>
> Quellenangaben unter Quellen.

3. Prinzip 2: Diversität — Vermeidung von Kettenfehlern

Während Redundanz Mehrfachvorhaltung adressiert, zielt Diversität darauf ab, **gemeinsame Fehlerursachen** (Common Mode Failures) zu vermeiden. Systeme, die redundant, aber homogen sind, teilen dieselben Schwachstellen und können daher simultan scheitern.

3.1 Formen technischer Diversität

- **Komponentendiversität**
  - (unterschiedliche HSMs, unterschiedliche CA-Stacks, verschiedene Backup-Mechanismen)
- **Lieferantendiversität**
  - (Multi-Vendor-Firewalls, Multi-Cloud-Strategien, verschiedene Identity-Provider)
- **Technologiendiversität** (unterschiedliche Betriebssysteme, Container vs. VM, heterogene Storage-Systeme)

Diversität operiert damit auf einer Metaebene: Sie reduziert die Wahrscheinlichkeit synchroner Ausfälle durch **Nicht-Korrelation von Fehlerursachen**.

Ergänzung zur Vorlesung: CrowdStrike 2024

Am 19. Juli 2024 verteilte CrowdStrike ein fehlerhaftes Konfigurations-Update für seinen Windows-Sensor. Nach Angaben von Microsoft stürzten rund 8,5 Mio. Windows-Rechner ab – weniger als 1 % aller Windows-Geräte, aber konzentriert in Unternehmen mit hohem Schutzbedarf. Flughäfen, Krankenhäuser, Banken und Sender standen still; viele Rechner mussten von Hand repariert werden. Hochverfügbare Cluster halfen nicht, weil alle Knoten denselben Agenten verwendeten: ein Common-Mode-Failure im Lehrbuchsinn.

Der Fall ergänzt die drei Formen der Diversität um eine vierte, die **zeitliche Diversität**: Gestaffelte Rollouts (Canary) verteilen eine Änderung erst auf wenige Systeme und dann auf alle, sodass ein Fehler nicht überall gleichzeitig wirkt. CrowdStrike hat als Konsequenz eine solche gestaffelte Auslieferung angekündigt.

Quellenangaben unter Quellen.

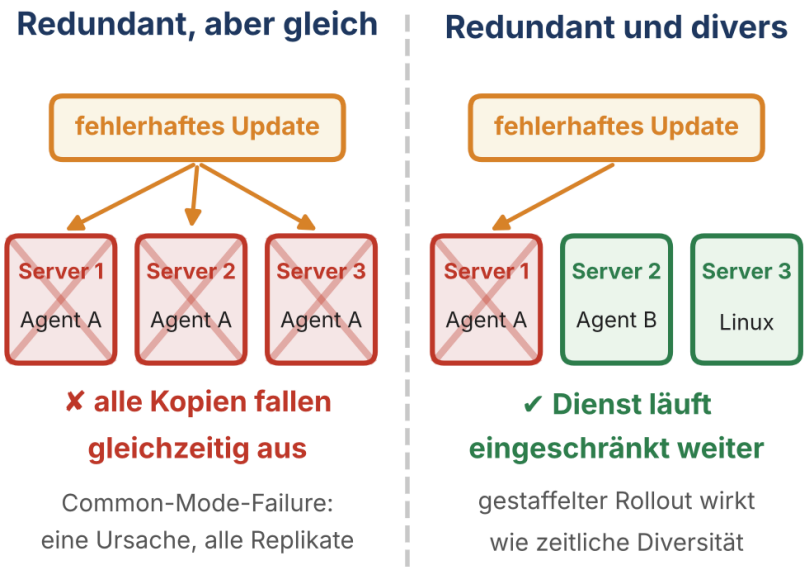


Abbildung 2: Redundant, aber gleich gegenüber redundant und divers

3.2 Diversität als Resilienz gegen Supply-Chain-Risiken

Supply-Chain-Angriffe (z. B. SolarWinds 2020) zeigen, dass Homogenität auf Prozess- und Hersteller-Ebene ein systemischer Verstärker ist.

Ein divers gelagerter Technologie-Stack wirkt wie ein „Brandschott“ gegen seitliche Ausbreitung (vgl. ENISA-Supply-Chain-Guidance).

3.3 Kosten- und Komplexitätsaspekte

Diversität erhöht Betriebskosten (mehr Know-how, mehr Verträge, mehr Integrationsaufwand).

In der Resilienzperspektive ist Diversität jedoch ein **Versicherungspotenzial** gegen Low-Probability–High-Impact-Ereignisse.

NIST SP 800-160 Vol. 2 und ISO/IEC 27031 klassifizieren Diversität daher explizit als Architekturprinzip für Resilience Engineering — nicht als Kostenfaktor, sondern als **Risikodiffusionsmechanismus**.

4. Prinzip 3: Wiederherstellbarkeit — Recovery als geübte Fähigkeit

Wiederherstellbarkeit ist das zentrale Bindeglied zwischen messbarer Resilienz (vgl. RTO/RPO) und tatsächlicher Systemfähigkeit. Ein System ist nicht resilient, weil es redundante Daten besitzt, sondern weil es nach einem Störfall **kontrolliert und innerhalb akzeptabler Grenzwerte** wieder in den Betrieb überführt werden kann.

4.1 Recovery ist eine Fähigkeit, kein Dokument

Viele Organisationen besitzen umfangreiche Notfallhandbücher, aber keine geübten Wiederanlaufprozesse.

BSI-Standard 200-4 und ISO/IEC 27031 betonen daher die Notwendigkeit von:

- **Recovery-Tests** (nicht nur Planexistenz)
- **regelmäßigen Notfallübungen**
- **Technik-/Prozess-Integration** (nicht isolierte Dokumentation)
- **Lessons Learned-Zyklen** zur Anpassung der Pläne

Ein ungeübter Plan ist funktional äquivalent zu keinem Plan.

4.2 Technische Realisierung von Wiederherstellbarkeit

In modernen Architekturen erfolgt Recovery zunehmend automatisiert:

- **Infrastructure as Code (IaC)** ermöglicht reproduzierbare Neuaufbauten
- **Snapshots / Immutable Backups** reduzieren RPO-Verluste
- **Runbooks & SOAR-Automatisierung** verkürzen MTTR
- **Failover-Szenarien** (On-Prem→Cloud oder Cross-Region) minimieren RTO

Wesentlich ist nicht die Existenz dieser Mittel, sondern die **Validierung durch Test und Metrik** (vgl. NIST SP 800-160 Vol. 2).

4.3 Abgrenzung: Backup ≠ Recovery

- **Backup** ist ein Zustand (gespeicherter Datenbestand).
- **Recovery** ist ein Prozess mit Zeit- und Qualitätszielen (RTO/RPO).
- **Ungetestetes Backup** ist kein Nachweis — sondern eine Hypothese.

Die häufigsten Resilienzversagen resultieren nicht aus fehlenden Daten, sondern aus **fehlgeschlagenen Wiederanläufen** infolge ungeprüfter Abhängigkeiten, mangelhafter Automatisierung oder fehlender Rollenklärung.

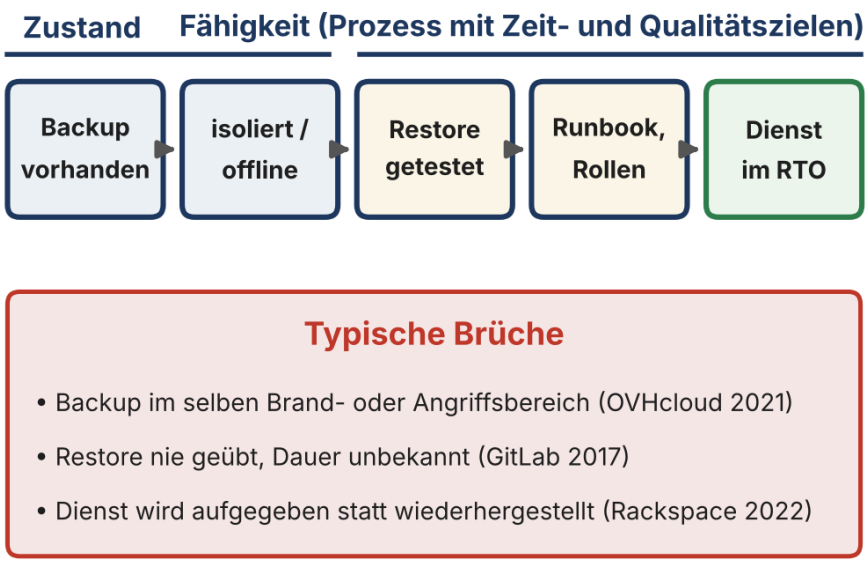


Abbildung 3: Recovery als Kette: Nur das erste Glied ist ein Zustand, alle weiteren sind Fähigkeiten

Ergänzung zur Vorlesung: Rackspace 2022

Im November 2022 veröffentlichte Microsoft einen Patch für die Exchange-Lücke „ProxyNotShell". Der Hosting-Anbieter Rackspace spielte ihn aus Sorge vor Anmeldeproblemen nicht ein und verließ sich auf vorläufige Workarounds. Am 2. Dezember 2022 umging die Ransomware-Gruppe Play diese Workarounds und legte den Dienst Hosted Exchange lahm. Anfang Januar 2023 entschied Rackspace, die Umgebung nicht wieder aufzubauen, sondern den Dienst einzustellen und die Kunden zu Microsoft 365 zu migrieren.

Für die Kunden war das RTO des alten Dienstes damit faktisch unendlich. Der Fall zeigt, dass Recovery nicht nur eine technische, sondern auch eine wirtschaftliche Entscheidung ist – und dass Kunden eines Dienstleisters eine eigene Exit- und Wiederherstellungsstrategie brauchen.

Quellenangaben unter Quellen.

5. Prinzip 4: Adaptivität — Resilienz als dynamische Systemeigenschaft

Adaptivität unterscheidet resiliente Systeme von stabilen Systemen. Stabilität beschreibt das Halten eines Zustands; Resilienz beschreibt die Fähigkeit, sich unter Störung **neu zu konfigurieren**, ohne Funktionsverlust als Ganzes. Adaptivität ist damit eine aktive Resilienzleistung — kein statisches Designmerkmal.

5.1 Formen adaptiver Resilienzmechanismen

- **Auto-Scaling** zur Kompensation von Lastspitzen oder Knotenverlusten
- **Graceful Degradation** (Funktionsreduktion ohne Totalausfall)
- **Feature Flags / Circuit Breaker / Canary Releases** zur kontrollierten Drosselung
- **Traffic Shifting / Anycast / Failover Logic** zur Neuverteilung von Last oder Regionen

Diese Mechanismen verhindern binäre Zustandswechsel („an/aus“) und erzeugen „abgestufte Überlebensfähigkeit“.

5.2 Adaptivität als trainierte Eigenschaft (Chaos Engineering)

Cloud-native Resilience Engineering setzt zunehmend auf **kontrollierte Störinjektion** (z. B. Netflix „Chaos Monkey“), um Systeme adaptives Verhalten nicht nur zu entwerfen, sondern zu testen. Diese Praxis folgt dem Grundsatz:

Resilienz entsteht nicht durch Erwartung, sondern durch Exposition.

NIST SP 800-160 Vol.2 ordnet adaptives Verhalten den Fähigkeiten „respond“, „recover“ und „adapt“ zu und positioniert es als lernfähigen Prozess statt als statischen Architekturstatus.

> Ergänzung zur Vorlesung: Netflix und der AWS-Ausfall 2011
>
> Am 21. April 2011 war eine Zone der AWS-Region US-East gestört; viele bekannte Dienste waren offline. Netflix lief weiter, wenn auch mit höherer Fehlerrate und Latenz. Gründe laut Netflix: zustandslose Dienste, Verteilung auf drei aktive Zonen und vorgehaltene Reservekapazität. Chaos Monkey hatte Instanzausfälle zur Routine gemacht.
>
> Netflix benannte aber auch die Lücke: Einen Ausfall einer ganzen Zone hatte Chaos Monkey nicht simuliert, die Umleitung lief deshalb teilweise manuell. Daraus entstanden weitere Werkzeuge, die gezielt größere Ausfälle üben. Adaptivität ist also nur so gut wie die Störungen, mit denen sie trainiert wurde.
>
> Quellenangaben unter Quellen.

5.3 Grenzen der Adaptivität

Adaptivität kann Redundanz und Diversität nicht ersetzen. Systeme, die nur adaptiv sind, aber auf einem gemeinsamen Identitäts- oder Datensinglepoint aufbauen, adaptieren lediglich **innerhalb der Fragilität**.

Adaptivität ist deshalb ein Multiplikator, kein Ersatz für die anderen Prinzipien.

6. Hybridarchitekturen: Chancen, Risiken und SPOF-Dynamiken

Hybride IT-Landschaften kombinieren klassische On-Premises-Systeme mit Cloud-Diensten. Aus Resilienzsicht entsteht dabei weder automatisch ein Vorteil noch ein Risiko — entscheidend ist, **wo Abhängigkeiten liegen**. Hybrid erhöht stets die Zahl der Komponenten, aber nicht zwangsläufig die Zahl der Ausfallpfade.

6.1 Resilienzpotenziale im Hybridbetrieb

- **Workload-Verlagerung** (Failover von On-Prem in Cloud / umgekehrt)
- **Diversifizierung der Ausführungsumgebung** → Schutz gegen Standort- und Lieferantenrisiken
- **Entkopplung über APIs / Message-Bus / Asynchronität**
- **Resilienz „as a Service“ konsumierbar** (z. B. Multi-AZ, Geo-Replication)

Hybrid ermöglicht damit, Resilienz **funktional** zu implementieren statt rein physisch.

6.2 Hybrid-SPOFs — neue Fragilitätsquellen

Viele hybride Systeme scheitern nicht an Rechen- oder Speicherressourcen, sondern an **zentralen Kopplungspunkten**, u. a.:

- **Identity-Systeme (AD/IdP)** als globaler Enabler → einheitlicher Fail-Knopf
- **Netzwerk-Transit** (VPN/SD-WAN) als Engpass
- **Zentrale Orchestrierung/CI/CD-Pipelines** als Single-Control-Layer
- **Datenkopplung in Echtzeit** → Resilienzverlust durch enge Synchronisationszwänge

Ein System kann funktional verteilt sein, aber steuerungstechnisch monolithisch — und damit fragil.

> Ergänzung zur Vorlesung: Facebook 2021
>
> Am 4. Oktober 2021 trennte ein Wartungsbefehl versehentlich alle Verbindungen im Backbone von Facebook; ein Fehler im Prüfwerkzeug verhinderte, dass er gestoppt wurde. Die DNS-Server zogen daraufhin – wie vorgesehen – ihre BGP-Routen zurück. Facebook, Instagram und WhatsApp waren rund sechs Stunden weltweit nicht erreichbar. Auch interne Werkzeuge und der Fernzugriff fielen aus, sodass Techniker vor Ort in die Rechenzentren mussten; die strengen Zutrittsregeln verzögerten die Wiederherstellung zusätzlich.
>
> Der Fall zeigt den Hybrid-SPOF im großen Maßstab: Tausende redundante Server, aber eine gemeinsame Steuerungsebene. Zugleich wird sichtbar, dass Sicherheitsmaßnahmen die Wiederherstellung verlangsamen können.
>
> Quellenangaben unter Quellen.

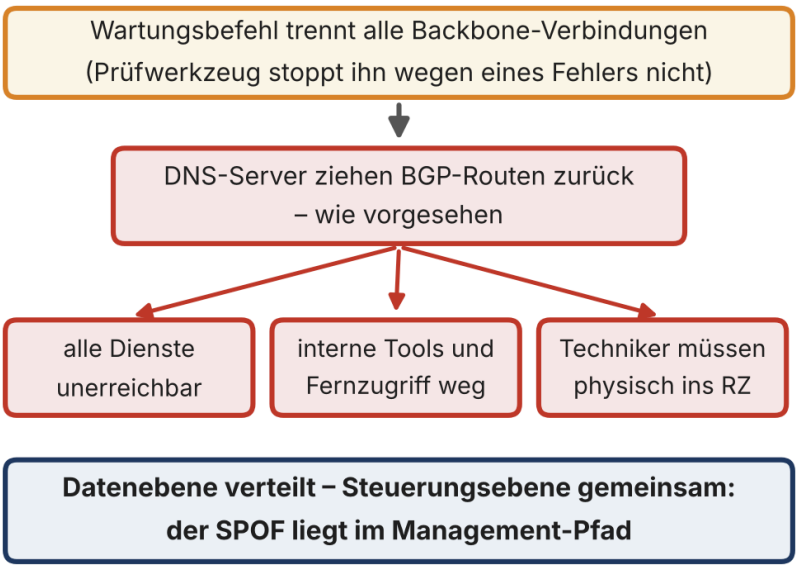


Abbildung 4: Facebook 2021: Die Datenebene war verteilt, die Steuerungsebene gemeinsam

6.3 Hybrid verlangt doppelte Resilienzlogik

- Alles, was „On-Prem resilient“ sein muss, muss **auch für Cloud-Anteile gelten**
- „Cloud Resilience“ ersetzt **nicht** RZ-Resilienz, sondern ergänzt sie
- **SPOF-Suche muss über Systemgrenzen hinweg erfolgen** (Identity, Netzwerk, Control-Plane)

Hybrid erhöht also den **Suchraum der Fragilität**, aber gleichzeitig den **Werkzeugkasten der Resilienz**. Resilienzgewinne entstehen nur, wenn die neuen Freiheitsgrade bewusst genutzt werden.

7. Übung: SPOF-Analyse und Prinzip-Zuordnung (Gruppenarbeit)

Die Übung dient der Anwendung der vier technischen Resilienzprinzipien (Redundanz, Diversität, Wiederherstellbarkeit, Adaptivität) auf reale Systeme. Ziel ist nicht die vollständige Analyse, sondern das **Erkennen struktureller Verwundbarkeit**.

Aufgabenstellung (20–30 Minuten in Gruppen):

1. **System wählen**
   - z. B. Cloud-Drive, Banking-App, Campus-E-Mail, Smart-Home-Plattform, Krankenhaus-IT
2. **Mindestens drei SPOFs identifizieren**
   - (z. B. IdP, Netzwerk-Backbone, zentrales Storage, CI/CD-Pipeline)
3. **Jedem SPOF ein Resilienzprinzip zuordnen**, das ihn adressieren würde — Redundanz / Diversität / Wiederherstellbarkeit / Adaptivität
4. **Eine realistische Maßnahme pro SPOF formulieren** (keine Wunschlösung, sondern praktisch plausibel)

Ergebnisformat: 1 Slide oder 5-Punkte-Statement

Die Übung folgt der Logik: Resilienz beginnt nicht mit Technik, sondern mit dem **Benennen des Bruchpunkts**.

8. Zusammenfassung & Prüfungsrelevanz

Diese Sitzung hat gezeigt, dass technische Resilienz nicht durch eine einzelne Maßnahme entsteht, sondern durch das **Zusammenspiel von vier Architekturprinzipien**. Redundanz verhindert Einzelbruchpunkte, Diversität verhindert Kettenfehler, Wiederherstellbarkeit macht Ausfälle reversibel und Adaptivität reduziert den Wirkungsverlust in Echtzeit.

In hybriden Architekturen erhöht sich der Suchraum für Verwundbarkeit — aber auch der Werkzeugkasten für Resilienz.

Prüfungsrelevante Leitfragen:

- Erklären Sie den strukturellen Unterschied zwischen Redundanz und Diversität.
- Warum ist Recovery eine Fähigkeit und kein Plan? (vgl. ISO/IEC 27031; BSI-200-4)
- Nennen Sie zwei typische Hybrid-SPOFs und ordnen Sie je ein Gegenprinzip zu.
- Welche Rolle spielt Adaptivität im Unterschied zu Redundanz?

Normative Referenzen im Skriptkontext: NIST SP 800-160 Vol. 2 (Resilience Engineering), ISO/IEC 27031 (ICT Readiness for BC), BSI-200-4 (BCM-Reifegrad), ENISA-Guidance (Supply-Chain-Resilience).

Bibliografie